PROJECT DOCUMENTATION

FinBot AI – Personal Finance Advisor Bot

AI-Powered Personal Finance Management Web Application

Name : Dhanashri Haridas Misal

Project Repository: https://github.com/dhanashrimisal851-ai/FinBot-AI-

Abstract

FinBot AI (Personal Finance Advisor Bot) is a web-based personal finance management system designed to help users record income and expenses, plan budgets, monitor savings, and understand their financial habits. The application combines a Python Flask backend, an SQLite database managed through SQLAlchemy, interactive visualizations using Chart.js, and an AI financial advisor powered by Google Gemini. If an AI API key is not configured or the external call fails, a rule-based fallback engine can provide structured guidance from the user's stored financial data. The goal is to make everyday money management easier through organized records, clear dashboards, and practical, understandable suggestions.

Chapter 1: Introduction

1.1 General Information / Topic

Project title: FinBot AI – Personal Finance Advisor Bot. The project belongs to the domains of personal finance management, web application development, and artificial intelligence. It provides a single interface for maintaining financial transactions, setting budgets and savings goals, and viewing summarized financial reports.

1.2 Problem Statement

Many people track their money informally or across separate notes and spreadsheets. This can make it difficult to understand spending patterns, identify overspending, maintain a budget, and measure progress toward savings goals. Users may also find financial information difficult to interpret without clear summaries.

1.3 Explanation of the Problem

Without a centralized system, users may not have an accurate view of income, expenses, remaining budget, or monthly savings. Manual calculations take time and can lead to mistakes. A useful solution should store transactions in an organized way, summarize them automatically, display trends visually, and provide suggestions based on the available records.

1.4 Targeted Community

- Students managing allowances, scholarships, or part-time income.
- Salaried employees tracking monthly income and household expenses.
- Freelancers with irregular or fluctuating income.
- Individuals and households who want a simple way to plan budgets and savings.

1.5 Aim of the Project

To develop a user-friendly web application that helps users manage personal finances through transaction tracking, budget planning, savings goals, visual reports, and AI-assisted financial guidance.

1.6 Objectives of the Project

- Provide secure user registration, login, logout, and profile management.
- Allow users to record, categorize, search, filter, and delete income and expense entries.
- Display key financial indicators and charts on a dashboard.
- Support monthly budgets and category-specific spending limits.
- Help users create savings goals and record contributions.
- Generate monthly/yearly reports and allow CSV export.
- Offer AI-generated explanations and recommendations with a rule-based fallback.
- Keep the application modular so features can be tested and maintained.

Chapter 2: Literature Review

2.1 Existing Approaches

Common personal finance approaches include paper records, spreadsheets, banking applications, and dedicated budgeting tools. Paper and spreadsheet methods are flexible but require manual updates and calculations. Banking applications can summarize transactions from supported accounts, but may not support every income source or the user's preferred budgeting method. A standalone finance manager can offer customizable categories, goals, and reporting in one place.

2.2 Proposed Solution

FinBot AI combines transaction management, budget monitoring, savings goals, reports, visual charts, and a conversational advisor in a single web application. It supports different financial profiles, including student, salaried, freelancer, and household modes.

2.3 References / Technologies Consulted

- Flask documentation: https://flask.palletsprojects.com/
- Flask-SQLAlchemy documentation: https://flask-sqlalchemy.palletsprojects.com/
- SQLAlchemy documentation: https://docs.sqlalchemy.org/

- Google Gemini API documentation: https://ai.google.dev/
- Chart.js documentation: https://www.chartjs.org/docs/
- pytest documentation: https://docs.pytest.org/

Chapter 3: System Analysis

3.1 Existing System

In a manual process, users enter financial information in notebooks or spreadsheets and calculate totals themselves. This approach can be time-consuming and may not provide integrated reminders, interactive charts, goal progress, or contextual AI assistance.

3.2 Proposed System

The proposed system stores each user's financial information in a database, calculates summaries, compares spending with budgets, tracks savings contributions, and presents information through a responsive browser interface. The AI advisor uses available financial context to generate explanations and suggestions.

3.3 Functional Requirements

- User registration, login, logout, and session-protected pages.
- Profile and financial mode selection.
- Income and expense creation, viewing, filtering, sorting, and deletion.
- Dashboard summaries and interactive charts.
- Monthly and category-level budget management.
- AI-assisted budget allocation and financial Q&A.
- Savings goal creation, progress display, and contribution logging.
- Monthly/yearly reporting, CSV download, and print-friendly layout.

3.4 Non-Functional Requirements

- Usability: clear navigation and understandable financial summaries.
- Security: password hashing, protected sessions, input validation, and careful handling of secret keys.
- Reliability: rule-based advisor fallback when the external AI service is unavailable.
- Maintainability: separation of routes, models, services, templates, and static assets.
- Performance: efficient database queries and appropriately sized visualizations.
- Portability: runs in a Python environment and can be deployed with a production WSGI server.

3.5 Hardware and Software Requirements

- Hardware: computer or laptop with a modern web browser and internet access for AI API use.

- Operating system: macOS, Linux, or Windows.
- Python: version 3.9 or later.
- Backend: Flask, Flask-SQLAlchemy, Flask-CORS, SQLAlchemy.
- Database: SQLite by default.
- Frontend: HTML5, CSS3, JavaScript, Tailwind CSS, Font Awesome, Chart.js.
- AI integration: Google Gemini API; rule-based fallback if not configured.
- Testing and serving: pytest and Gunicorn.

Chapter 4: System Design

4.1 System Architecture

The application follows a layered web architecture. The browser renders HTML templates and JavaScript interactions. Requests are handled by Flask routes and API blueprints. Route handlers use SQLAlchemy models and service modules for persistence and business logic. The SQLite database stores users, transactions, budgets, goals, contributions, and chat history. The AI service calls Gemini when configured and otherwise uses fallback rules.

4.2 Main Modules

- Authentication and Profiles: registration, login, logout, sessions, and financial modes.
- Dashboard: income, expenses, net savings, savings rate, remaining budget, and charts.
- Income Management: categorized income entries.
- Expense Management: categorized expenses, need/want tags, payment method, search, filters, and sorting.
- Budget Planner: overall and category budgets, spending progress, warnings, and allocation suggestions.
- AI Advisor: finance-context-aware chat with Gemini and fallback logic.
- Savings Goals: targets, current progress, estimated completion, and contributions.
- Reports: consolidated summaries, CSV export, and print layout.

4.3 Database Design

The project README identifies SQLAlchemy models for User, Income, Expense, Budget, BudgetCategory, SavingsGoal, SavingsContribution, and ChatHistory. In general, user-owned records should be associated with the relevant user so that each account accesses only its own data. Exact field names and relationships should be confirmed against the model source files.

4.4 Data Flow

- The user signs in and opens a module such as Expenses or Budgets.
- The frontend sends a request to a Flask route or API endpoint.

- The route validates input and invokes the appropriate service/model logic.
- SQLAlchemy reads or updates the SQLite database.
- The application returns a page or structured response to the browser.
- Dashboard scripts render charts from the returned financial summaries.
- For advisor questions, the AI service uses relevant stored financial context and returns a generated or rule-based response.

Chapter 5: Implementation Details

5.1 Technology Stack

- Python 3.9+: application logic and backend processing.
- Flask 3.0.3: web framework and routing.
- Flask-SQLAlchemy 3.1.1 / SQLAlchemy: database integration and ORM.
- SQLite: local relational database.
- HTML, CSS, JavaScript, Tailwind CSS: user interface.
- Chart.js: interactive financial charts.
- Google Gemini API: AI-based conversational financial guidance.
- pytest: automated tests.
- Gunicorn: production WSGI server.

5.2 Project Structure

The repository is organized into app.py and config.py for application setup; models/ for database models; routes/ for API blueprints; services/ for finance, budget, AI, and report logic; utils/ for helpers and validators; templates/ for Jinja2 pages; static/ for CSS and JavaScript; scripts/ for demo data; and tests/ for automated checks.

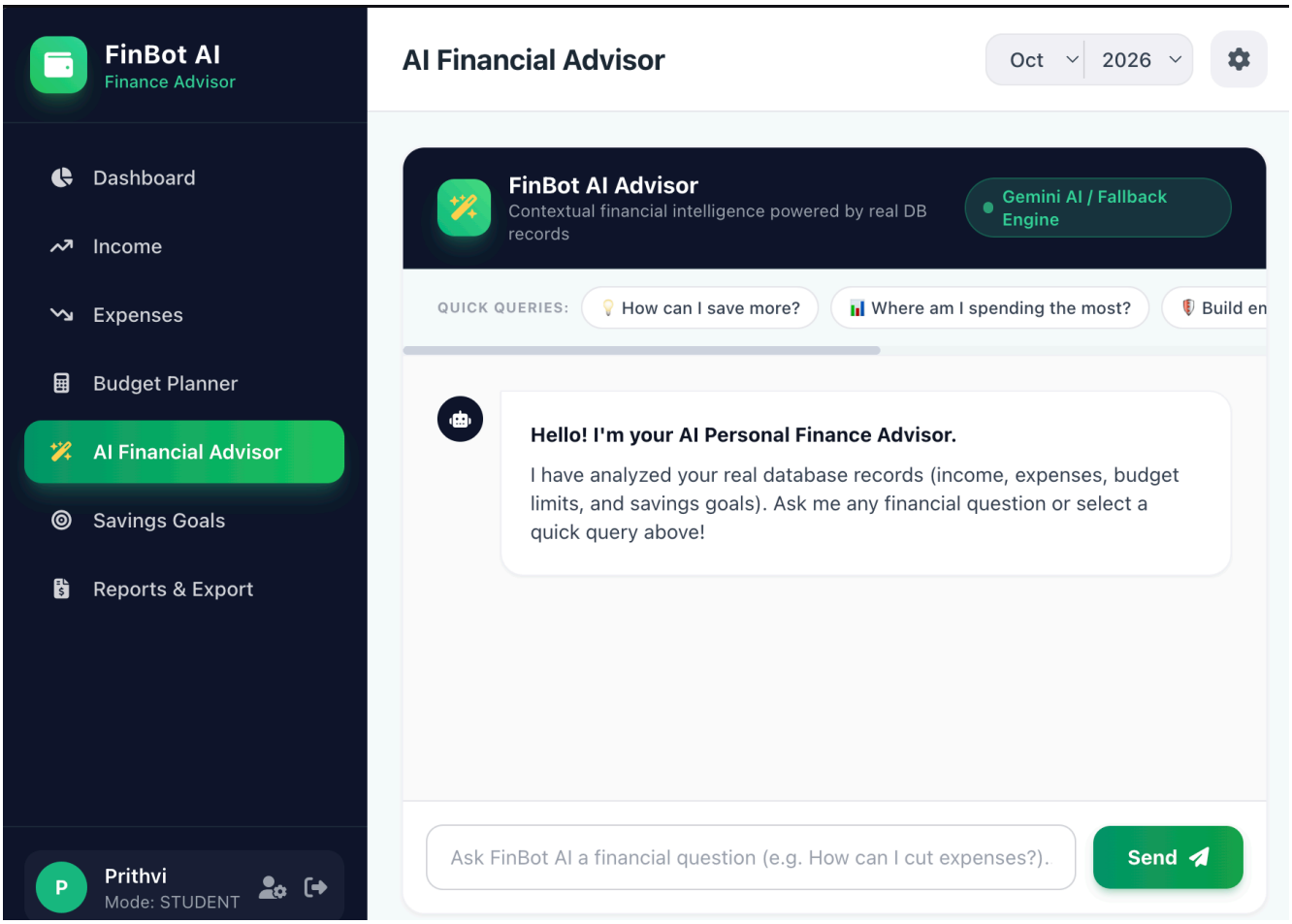
5.3 AI Advisor and Fallback

When GEMINI_API_KEY is configured, the advisor can send a question and relevant financial context to the Gemini service. If the key is absent or an API request fails, the fallback engine supplies structured recommendations using local application logic. AI-generated content should be treated as general educational guidance, not as a guarantee of financial results or a substitute for a qualified financial professional.

5.4 Security Considerations

- Use a strong, unique FLASK_SECRET_KEY and keep .env out of version control.
- Never publish API keys, passwords, or real user financial records.

- Use password hashing and session protection.
- Validate and sanitize user inputs on the server.
- Ensure database queries and reports are scoped to the authenticated user.
- Use HTTPS in production and review cookie/security settings before deployment.



Chapter 6: Installation and Execution

6.1 Prerequisites

Install Python 3.9 or later and Git. Download or clone the repository and open a terminal in the project directory.

6.2 Setup on macOS / Linux

```
git clone https://github.com/dhanashrimisal851-ai/FinBot-AI-.git

cd FinBot-AI-

python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt

cp .env.example .env
```

```
python scripts/seed_demo_data.py

python app.py
```

Then open http://127.0.0.1:5000 in a browser. If the repository's current files differ from these instructions, follow the actual README and configuration files in the checked-out branch.

6.3 Environment Variables

- FLASK_SECRET_KEY: secret used to secure Flask sessions.
- GEMINI_API_KEY: optional key for Gemini; leave blank to use fallback AI.
- GEMINI_MODEL: model identifier configured by the project.
- DATABASE_URL: database connection URL; SQLite is the default.

6.4 Running Tests

Run the automated test suite from the project root using: pytest

6.5 Production Deployment

For a production deployment, use a production WSGI server such as Gunicorn rather than Flask's development server. The repository documents a command similar to: gunicorn app:app --bind 0.0.0.0:5000. Configure environment variables securely, enable HTTPS, and consider PostgreSQL or another managed database for multi-user production workloads.

Chapter 7: Testing

7.1 Testing Strategy

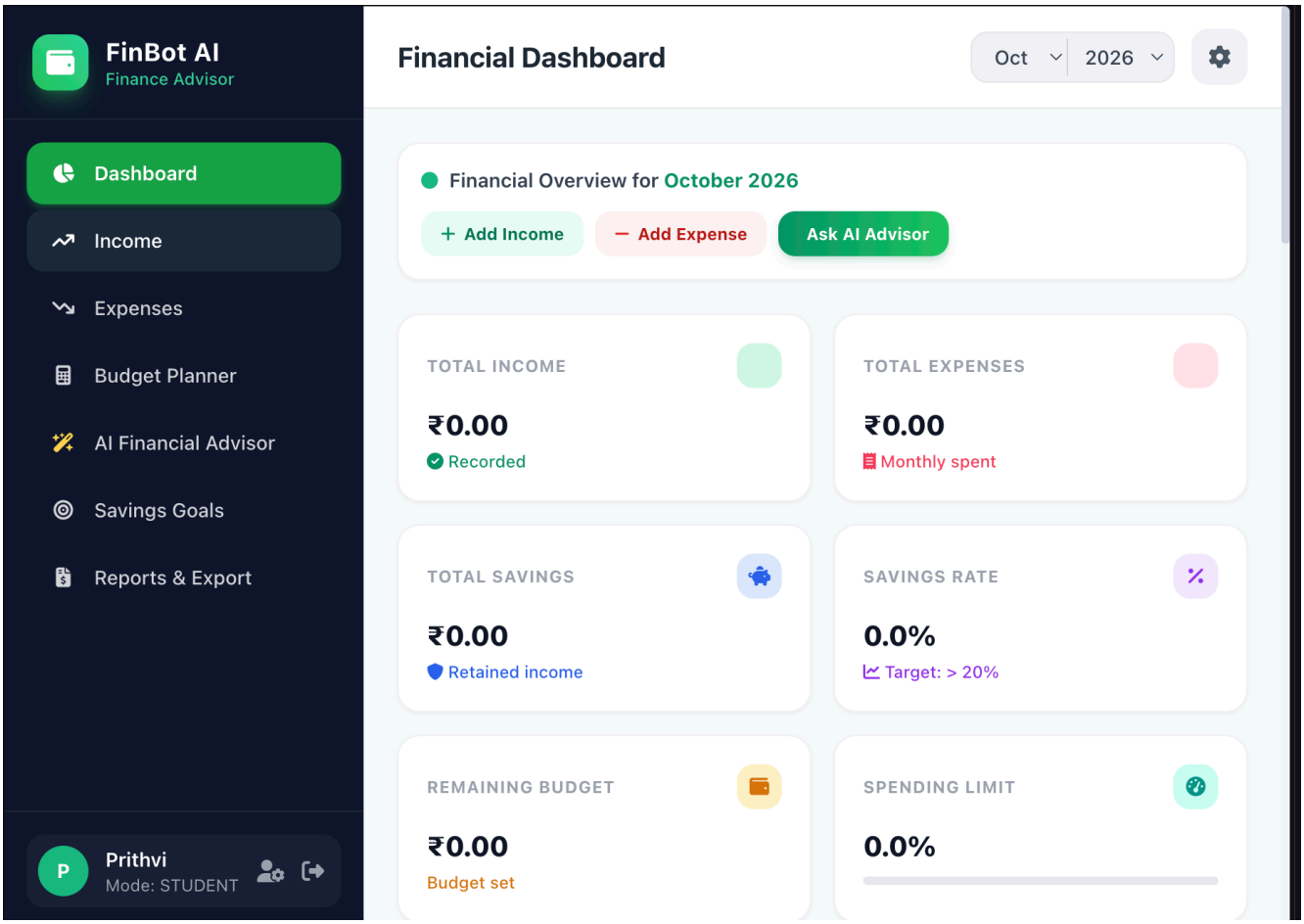
The repository includes pytest test modules covering authentication, finance calculations, budgets and goals, AI advisor behavior, and reports. Tests should be run after changes to confirm that important workflows continue to work.

7.2 Suggested Test Cases

1. Register with valid details and reject invalid or duplicate registration data.
2. Log in with correct credentials and reject incorrect credentials.
3. Create income and expense entries and confirm dashboard totals update.
4. Filter and sort expenses and confirm the expected records are shown.
5. Set a budget and confirm spending progress and over-budget warnings.
6. Create a savings goal, add a contribution, and verify progress changes.
7. Ask the advisor a question with Gemini configured and with the API key absent.
8. Generate a monthly report and verify the CSV file can be opened.
9. Verify that one user cannot access another user's financial records.

7.3 Expected Results

Valid operations should persist correctly and update the relevant summaries. Invalid inputs should be rejected with understandable feedback. Charts and reports should match the stored transaction data. If Gemini is unavailable, the fallback advisor should remain usable.



Chapter 8: Results and Discussion

The intended result is a single web application that brings together financial record keeping, budgeting, savings goals, visualization, and AI-assisted explanations. The dashboard helps users understand their current position, while reports and goal tracking support longer-term planning. The fallback AI path improves availability when the external model service cannot be used. Actual performance, usability, and correctness should be confirmed by running the application and recording test results on the target environment.

Chapter 9: Limitations and Future Scope

9.1 Limitations

- AI responses may be inaccurate or incomplete and should be checked before acting on them.
- SQLite is convenient for local use but may not be ideal for larger concurrent deployments.

- The application depends on users entering accurate and timely transactions.
- Automatic bank synchronization, payment execution, and verified investment recommendations are not claimed by this documentation.

9.2 Future Enhancements

- Add optional bank statement import with user consent and privacy safeguards.
- Introduce recurring transactions and budget reminders.
- Add stronger report filters and downloadable PDF reports.
- Add multi-currency support and improved accessibility.
- Add more automated tests, audit logging, and deployment monitoring.
- Evaluate AI responses with finance-specific test cases and display clear limitations.
- Use a managed production database and regular backup strategy when scaling.

Chapter 10: Conclusion

FinBot AI is designed to make personal finance management more organized and accessible. By combining income and expense tracking, budget planning, savings goals, charts, reporting, and an AI advisor, it helps users review their financial habits in one place. The modular Flask architecture supports future improvements, while the rule-based fallback reduces dependence on an external AI service. Before production use, security, data privacy, financial calculations, and AI output quality should be thoroughly tested.

References

- FinBot AI project repository: https://github.com/dhanashrimisal851-ai/FinBot-AI-
- Flask: https://flask.palletsprojects.com/
- Flask-SQLAlchemy: https://flask-sqlalchemy.palletsprojects.com/
- SQLAlchemy: https://docs.sqlalchemy.org/
- Google AI for Developers: https://ai.google.dev/
- Chart.js: https://www.chartjs.org/docs/
- pytest: https://docs.pytest.org/

Appendix: Project Metadata

Project Name: FinBot AI – Personal Finance Advisor Bot

Project Type: AI-enabled web application

Primary Language: Python

Database: SQLite (default configuration)

Repository: https://github.com/dhanashrimisal851-ai/FinBot-AI-

Note: This document is based on the repository README. Verify implementation-specific details, screenshots, actual test outputs, and institution-specific formatting against the current source code before final submission.

